

PROGRAMMING IN HASKELL



Capítulo 3 - Tipos e Classes

O que é um Tipo?

Um tipo (type) é o nome que se dá para uma coleção de valores relacionados. Por exemplo, o tipo básico

Bool

contém dois valores lógicos:

False

True

Erros de Tipo

Aplicar uma função a um ou mais argumentos do tipo incorreto leva a um erro de tipo (type error).

```
> 1 + False  
error ...
```

1 é um número e False é um valor lógico, porém + requer dois números.

Tipos em Haskell

- Se a avaliação de uma expressão e produziria um valor do tipo t , então dizemos que e é do tipo t , anotado

$e :: t$

- Toda expressão bem formada tem um determinado tipo, que pode ser computado automaticamente em tempo de compilação usando um procedimento chamado inferência de tipo.

- Todos erros de tipo são encontrados em tempo de compilação, o que torna os programas mais seguros e eficientes já que a checagem de tipo não acontece em tempo de execução.
- No GHCi, o comando `:type` (ou `:t`) calcula o tipo de uma expressão, sem que a mesma seja avaliada. `:type +d` informa os tipos default associados às classes.

```
> not False  
True
```

```
> :type not False  
not False :: Bool
```

```
> :t (+)  
True  
(+) :: Num a => a -> a -> a  
> :t +d (+)  
(+) :: Integer -> Integer -> Integer
```

Tipos Básicos

Haskell possui vários tipos básicos, incluindo:

Bool

- valores lógicos

Char

- caracteres

String

- sequências de caracteres

Int

- números inteiros

Float

- números de ponto flutuante

Listas

Uma lista é uma sequência de valores do mesmo tipo:

```
[False, True, False] :: [Bool]
```

```
['a', 'b', 'c', 'd'] :: [Char]
```

Geralmente:

`[t]` é o tipo da lista com elementos do tipo `t`.

Notas:

- O tipo da lista não diz nada sobre seu tamanho:

```
[False,True] :: [Bool]
```

```
[False,True,False] :: [Bool]
```

- Não existe restrição sobre o tipo dos elementos da lista. Por exemplo, é possível ter uma lista de listas de caracteres:

```
[['a'],['b','c']] :: [[Char]]
```


Tuplas

Uma tupla é uma sequência de valores de tipos possivelmente diferentes:

```
(False, True) :: (Bool, Bool)
```

```
(False, 'a', True) :: (Bool, Char, Bool)
```

Geralmente:

$(t_1, t_2, \dots, t_n)$ é o tipo de tuplas de tamanho n cujo i -ésimo element possui tipo t_i para qualquer i em $1 \dots n$.

Notas:

- O tipo da tupla revela o tamanho da mesma:

```
(False, True) :: (Bool, Bool)
```

```
(False, True, False) :: (Bool, Bool, Bool)
```

- O tipo dos elementos da tupla é irrestrito:

```
('a', (False, 'b')) :: (Char, (Bool, Char))
```

```
(True, ['a', 'b']) :: (Bool, [Char])
```

Funções

Uma função é o mapeamento de valores de um tipo para valores de outro tipo:

```
not :: Bool → Bool
```

```
even :: Int → Bool
```

Geralmente:

$t1 \rightarrow t2$ é o tipo das funções que mapeiam valores do tipo $t1$ em valores do tipo $t2$.

Notas:

- A seta $\rightarrow$ representa os caracteres $\rightarrow$.
- O tipo dos argumentos e dos resultados são irrestritos. Por exemplo, funções com múltiplos argumentos ou múltiplos resultados são possíveis usando listas ou tuplas:

```
add :: (Int,Int)  $\rightarrow$  Int  
add (x,y) = x+y
```

```
zeroto :: Int  $\rightarrow$  [Int]  
zeroto n = [0..n]
```

Funções Currificadas (Curried)

Funções com múltiplos argumentos podem ser descritas através do retorno de funções como resultado:

```
add' :: Int → (Int → Int)
add' x y = x+y
```

`add'` recebe um inteiro `x` e retorna uma função `add' x`.
Por sua vez, essa função recebe um inteiro `y` e retorna o resultado `x+y`.

Notas:

- `add` e `add'` produzem o mesmo resultado final, mas `add` toma dois argumentos ao mesmo tempo, enquanto `add'` toma um de cada vez:

```
add :: (Int,Int) → Int
```

```
add' :: Int → (Int → Int)
```

- Funções que tomam um argumento por vez são chamadas de funções currificadas, em homenagem a Haskell Curry.

- Funções com mais de dois argumentos podem ser currificadas através do retorno de funções aninhadas:

```
mult :: Int → (Int → (Int → Int))  
mult x y z = x*y*z
```

`mult` toma um inteiro `x` e retorna a função `mult x`, a qual toma um inteiro `y` e retorna a função `mult x y`, que finalmente toma um inteiro `z` e retorna o resultado `x*y*z`.

Qual o motivo de usar currificação?

Funções currificadas são mais flexíveis que funções sobre tuplas, já que funções úteis podem ser criadas pela aplicação parcial de uma função currificada.

Por exemplo:

```
add' 1 :: Int → Int
```

```
take 5 :: [Int] → [Int]
```

```
drop 5 :: [Int] → [Int]
```


Convenções para currficação

Adotamos duas convenções para evitar o uso excessivo de parênteses quando usamos funções currficadas:

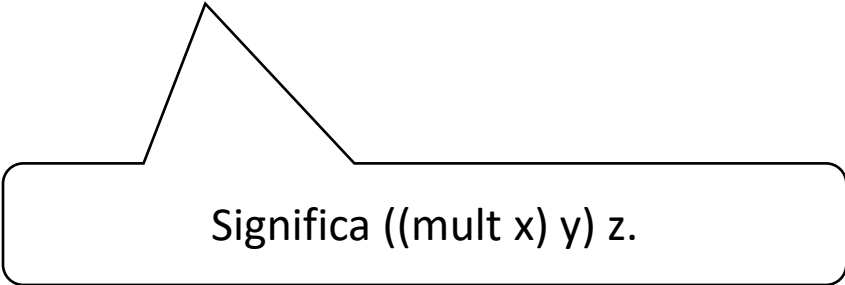
- A seta $\rightarrow$ é associativa à direita.

$\text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$

Significa $\text{Int} \rightarrow (\text{Int} \rightarrow (\text{Int} \rightarrow \text{Int}))$.

- As funções são aplicadas de forma associativa à esquerda.

`mult x y z`



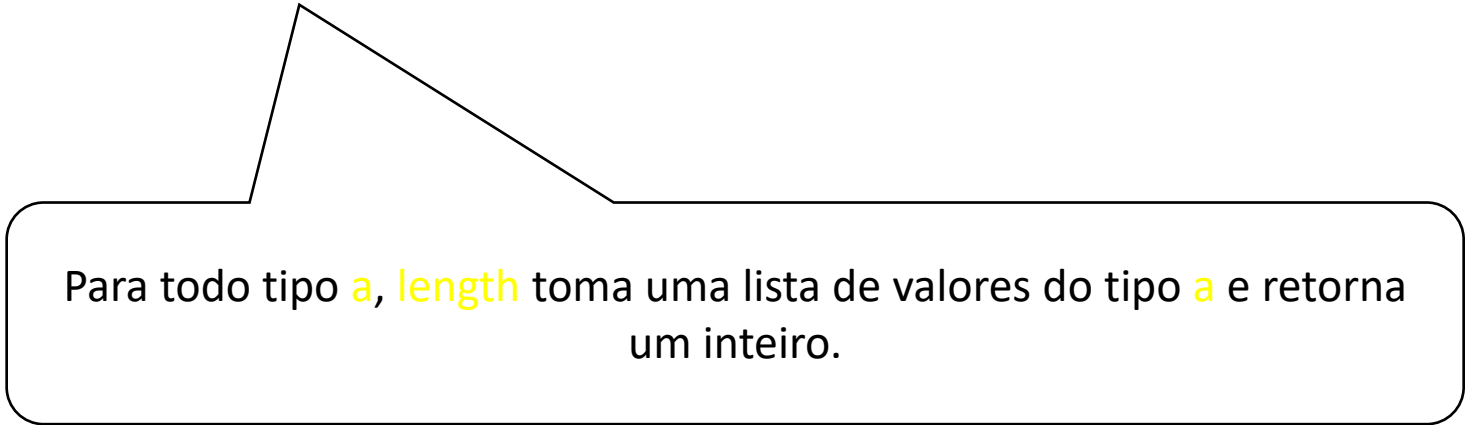
Significa $((\text{mult } x) y) z$.

A menos que o uso de tuplas seja explicitamente necessário, as funções em Haskell são usualmente definidas de forma currificada.

Funções Polimórficas

Uma função é dita polimórfica se o seu tipo contém uma ou mais variáveis de tipo.

```
length :: [a] → Int
```



Para todo tipo `a`, `length` toma uma lista de valores do tipo `a` e retorna um inteiro.

Notas:

- Variáveis de tipo podem ser instanciadas para diferentes tipos em diferentes circunstâncias:

```
> length [False,True]  
2
```

a = Bool

```
> length [1,2,3,4]  
4
```

a = Integer

- Variáveis de tipo devem começar com letra minúscula, e são geralmente nomeadas a, b, c, etc.

- Muitas funções definidas no standard prelude são polimórficas. Por exemplo:

```
fst :: (a,b) → a
```

```
head :: [a] → a
```

```
take :: Int → [a] → [a]
```

```
zip :: [a] → [b] → [(a,b)]
```

```
id :: a → a
```

Funções Sobrecarregadas

Uma função polimórfica é dita sobrecarregada (overloaded) se o seu tipo possui uma ou mais restrições de classe.

$(+) :: \text{Num } a \Rightarrow a \rightarrow a \rightarrow a$

Para qualquer **tipo numérico** a , $(+)$ toma dois valores do tipo a e retorna um valor do tipo a .

Notas:

- Variáveis com restrições de tipo podem ser instanciadas em qualquer tipo que satisfaça as restrições:

```
> 1 + 2  
3
```

a = Integer

```
> 1.0 + 2.0  
3.0
```

a = Double

```
> 'a' + 'b'  
ERROR
```

Char não é um **tipo**
numérico

- Haskell possui várias classes de tipos (type classes), tais como:

Num - Tipos numéricos

Eq - Tipos comparáveis

Ord - Tipos ordenáveis

- Exemplos:

```
(+) :: Num a => a -> a -> a
```

```
(==) :: Eq a => a -> a -> Bool
```

```
(<) :: Ord a => a -> a -> Bool
```


Dicas

- Sempre que você definir uma função em Haskell, é útil começar pela escrita de seu respectivo tipo.
- É boa prática escrever o tipo de cada função dentro de um script.
- Ao declarar os tipos de funções polimórficas que usam números, tipos comparáveis ou ordenáveis, tome cuidado para incluir as restrições de classe.

Exercícios

(1) Quais os tipos dos seguintes valores?

```
['a','b','c']  
('a','b','c')  
[(False,'0'),(True,'1')]  
([False,True],['0','1'])  
[tail,init,reverse]
```

(2) Quais são os tipos das seguintes funções?

```
second xs = head (tail xs)  
swap (x,y) = (y,x)  
pair x y = (x,y)  
double x = x*2  
palindrome xs = reverse xs == xs  
twice f x = f (f x)
```

(3) Verifique suas respostas via GHCi.